فازبندی اجرایی و تسک‌بندی پروژه

سیستم آنالیز تاکتیکی نورو-سمبلیک — از سند فنی تا اجرا

نسخه: ۱.۰ **پیش‌نیاز:** سند فنی نسخه ۲.۰ (تأییدشده) **نحوه استفاده:** هر فاز شامل چند تسک است. هر تسک یک معیار پذیرش مشخص و یک دستور تست pytest دارد. تسک بعدی را فقط بعد از سبز شدن تست تسک قبلی شروع کنید.

نمای کلی فازها

فاز	عنوان	خروجی نهایی
۰	راه‌اندازی محیط و زیرساخت	محیط کاری آماده، تست اتصال به داده
۱	بارگذاری و آماده‌سازی داده خام	جدول SPADL + فریم‌های 360 برای تمام بازی‌های بارسا در فصل ۲۰۲۰/۲۱
۲	تفکیک سکانس‌های Build-up	مجموعه سکانس‌های برچسب‌خورده (مثبت/منفی)
۳	ساخت لایه Rule-based Feature Extractor	فایل Prolog Facts تولیدشده از داده
۴	فرمول‌بندی دانش تاکتیکی (bk.pl مشترک)	bk.pl کامل با مشارکت مستقیم شما
۵	تعریف بایاس و اجرای Popper	اولین اجرای موفق ILP با قوانین خام خروجی
۶	فیلتر، ارزیابی و تفسیر قوانین	قوانین نهایی با Support/Confidence/Lift
۷	نمایش بصری و گزارش نهایی	گزارش قابل‌فهم با نقشه زمین برای هر الگو
۸	مستندسازی و بسته‌بندی MVP	جمع‌بندی، مستندات، آماده برای فاز توسعه

فاز ۰ — راه‌اندازی محیط و زیرساخت

تسک ۰.۱ — راه‌اندازی پروژه Python و مدیریت پکیج

کار: نصب `uv`، ساخت پروژه، تعریف `pyproject.toml` با وابستگی‌های اصلی (statsbombpy, socceraction, kloppy, pandas, numpy, mplsoccer, pytest). **معیار پذیرش:** دستور `uv sync` بدون خطا اجرا شود و تمام پکیج‌ها import شوند. **تست:**

```python
def test_core_imports():
    import statsbombpy, socceraction, pandas, numpy, mplsoccer
    assert True
```

تسک ۰.۲ — نصب و تست زنجیره Prolog/ILP

کار: نصب Clingo، SWI-Prolog، و Popper (طبق مستندات رسمی Popper). تست اجرای یک مثال نمونه‌ی پیش‌فرض Popper (مثال‌های آموزشی repo خودش). **معیار پذیرش:** مثال نمونه Popper بدون خطا اجرا شود و خروجی قانون تولید کند. **تست:**

```python
def test_popper_installed():
    import subprocess
    result = subprocess.run(["python", "-m", "popper", "--help"], capture_output=True)
    assert result.returncode == 0
```

تسک ۰.۳ — تست دسترسی به دیتاست هدف

کار: با statsbombpy، اتصال به StatsBomb Open Data و دریافت لیست بازی‌های بارسلونا در `competition_id=11, season_id=90`.
معیار پذیرش: لیست بازی‌ها دریافت شود و شامل حداقل ۲۰ بازی بارسلونا باشد. **تست:**

```python
```

```python
def test_barca_matches_available():
    from statsbombpy import sb
    matches = sb.matches(competition_id=11, season_id=90)
    barca = matches[(matches['home_team'] == 'Barcelona') | (matches['away_team'] == 'Barcelona')]
    assert len(barca) >= 20
```

تسک ۰.۴ — تست دسترسی به داده 360

کار: برای حداقل یک بازی بارسا، بررسی وجود و بارگذاری موفق داده 360 (`sb.frames`). **معیار پذیرش:** حداقل یک بازی با داده 360 کامل پیدا و بارگذاری شود. **تست:**

```python
def test_360_data_available_for_at_least_one_match(barca_match_ids):
    from statsbombpy import sb
    found = False
    for match_id in barca_match_ids:
        try:
            frames = sb.frames(match_id=match_id)
            if len(frames) > 0:
                found = True
                break
        except Exception:
            continue
    assert found
```

فاز ۱ — بارگذاری و آماده‌سازی داده خام

تسک ۱.۱ — دانلود و کش کردن داده خام تمام بازی‌های بارسا

کار: اسکریپت `load_raw_data.py` که تمام بازی‌های بارسا در آن فصل را دانلود و به‌صورت محلی (Parquet/JSON) کش می‌کند تا هر بار از اینترنت نخواند. **معیار پذیرش:** فایل‌های کش‌شده برای همه بازی‌ها ایجاد شوند؛ اجرای دوم اسکریپت از کش بخواند نه از اینترنت. **تست:**

```python
def test_all_matches_cached(barca_match_ids, cache_dir):
    for match_id in barca_match_ids:
        assert (cache_dir / f"events_{match_id}.parquet").exists()
```

تسک ۱.۲ — تبدیل Event Data به فرمت SPADL

کار: با socceraction، تبدیل داده خام هر بازی به جدول SPADL استاندارد. **معیار پذیرش:** جدول SPADL برای هر بازی دارای ستون‌های الزامی (player_id, team_id, action_type, start_x, start_y, end_x, end_y, result) باشد و تعداد ردیف‌ها با تعداد Eventهای خام هم‌خوانی معقول داشته باشد. **تست:**

```python
def test_spadl_schema(sample_spadl_df):
    required_cols = {"player_id","team_id","type_name","start_x","start_y","end_x","end_y","result_name"}
    assert required_cols.issubset(sample_spadl_df.columns)
    assert len(sample_spadl_df) > 0
```

تسک ۱.۳ — پیوند فریم‌های 360 به رخدادهای SPADL

کار: merge کردن هر ردیف SPADL با فریم 360 متناظرش (بر اساس event_id مشترک)، طوری‌که موقعیت بازیکنان بدون توپ در لحظه هر اکشن قابل دسترسی باشد. **معیار پذیرش:** حداقل ۷۰٪ از اکشن‌های Build-up (پاس/دریبل) دارای فریم 360 متناظر باشند (باقی به‌دلیل محدودیت پوشش دوربین طبیعی است). **تست:**

```python
def test_360_merge_coverage(merged_df):
    build_up_actions = merged_df[merged_df["type_name"].isin(["pass", "dribble"])]
    coverage = build_up_actions["freeze_frame"].notna().mean()
    assert coverage >= 0.70
```

تسک ۱.۴ — تعریف هویت و نقش بازیکنان بارسا

کار: ساخت جدول مرجع `players_reference` شامل شناسه هویتی (`player_identity`) و نقش تاکتیکی (`has_role`) هر بازیکن اصلی بارسا در آن فصل (طبق بخش ۲.۶ سند فنی). **معیار پذیرش:** جدول شامل تمام بازیکنانی باشد که در حداقل ۵ بازی آن فصل حضور داشته‌اند؛ هر بازیکن دقیقاً یک نقش اصلی داشته باشد. **تست:**

```python
def test_players_reference_complete(players_reference_df, spadl_all_matches):
    frequent_players = spadl_all_matches["player_id"].value_counts()
    frequent_players = frequent_players[frequent_players >= 5].index
    assert set(frequent_players).issubset(set(players_reference_df["player_id"]))
```

فاز ۲ — تفکیک سکانس‌های Build-up

تسک ۲.۱ — تعریف قاعده شروع/پایان یک سکانس Build-up

کار: پیاده‌سازی `sequence_builder.py` — منطقی که مشخص می‌کند یک سکانس Build-up از کجا شروع و کجا تمام می‌شود (مثلاً از دریافت توپ توسط دروازه‌بان/مدافع تا خروج از یک‌سوم دفاعی یا از دست دادن توپ). **معیار پذیرش:** روی حداقل ۳ بازی نمونه اجرا شود و سکانس‌هایی با طول منطقی (نه ۱ اکشنی، نه کل نیمه) تولید کند. **تست:**

```python
```

```python
def test_sequence_length_reasonable(sample_sequences):
    lengths = [len(seq) for seq in sample_sequences]
    assert all(2 <= l <= 25 for l in lengths)
```

تسک ۲.۲ — برچسب‌گذاری پیامد هر سکانس (مثبت/منفی)

کار: برای هر سکانس، تعیین پیامد نهایی (از دست دادن توپ تحت فشار = منفی؛ خروج موفق از فشار = مثبت) طبق معیاری که در بخش ۲.۷ سند فنی تعریف شد (Target روی پیامد، نه صرف تکرار). **معیار پذیرش:** هر سکانس دقیقاً یک برچسب داشته باشد؛ نسبت مثبت به منفی به‌گونه‌ای نباشد که یک طرف کمتر از ۱۰٪ کل باشد (برای معناداری آماری). **تست:**

python

```python
def test_label_balance(labeled_sequences_df):
    ratio = labeled_sequences_df["label"].value_counts(normalize=True)
    assert ratio.min() >= 0.10
```

تسک ۲.۳ — استخراج سکانس‌های چندبازیکنی نامزد (۲ تا ۳ نفره)

کار: برای هر سکانس، شناسایی زیرمجموعه‌های ۲ تا ۳ بازیکنی که در آن سکانس تعامل مستقیم داشته‌اند (پاس، پوشش، حمایت) — طبق بخش ۲.۵ سند فنی — تا بعداً به Popper داده شوند. **معیار پذیرش:** برای حداقل ۵۰٪ سکانس‌ها، حداقل یک زیرمجموعه دونفره معتبر استخراج شود. **تست:**

python

```python
def test_multi_player_subsets_extracted(sequences_with_subsets):
    has_subset = sequences_with_subsets["player_pairs"].apply(lambda x: len(x) > 0)
    assert has_subset.mean() >= 0.50
```

فاز ۳ — ساخت لایه Rule-based Feature Extractor

تسک ۳.۱ — پیاده‌سازی Predicateهای هندسی پایه

کار: توابع Python برای محاسبه `half_space`، `zone14`، `under_pressure` و مفاهیم مشابه از روی مختصات SPADL و فریم 360 (طبق بخش ۲.۲ و ۴.۱ سند فنی). **معیار پذیرش:** هر تابع روی حداقل ۳ ورودی دستی (مختصات شناخته‌شده) خروجی درست و منطقی بدهد (تست واحد ساده). **تست:**

```python
def test_half_space_geometry():
    assert occupies_half_space(x=70, y=15) == "right_half_space"
    assert occupies_half_space(x=70, y=34) == "central_channel"
```

تسک ۳.۲ — تولید Fact های Prolog از یک بازی نمونه

کار: تبدیل خروجی توابع هندسی به فایل `.pl` واقعی با Predicateهای Prolog معتبر، برای یک بازی نمونه. **معیار پذیرش:** فایل تولیدشده در SWI-Prolog بدون خطای Syntax لود شود. **تست:**

```python
def test_generated_facts_load_in_prolog(facts_file_path):
    import subprocess
    result = subprocess.run(["swipl", "-g", f"consult('{facts_file_path}'), halt."], capture_output=True)
    assert result.returncode == 0
```

تسک ۳.۳ — تعمیم تولید Fact به کل دیتاست

کار: اجرای پایپ‌لاین فوق روی تمام بازی‌های بارسا، ادغام در یک یا چند فایل `.pl` نهایی. **معیار پذیرش:** فایل نهایی شامل Factهایی برای تمام سکانس‌های برچسب‌خورده در فاز ۲ باشد؛ تعداد Factهای هر نوع با تعداد سکانس‌های ورودی هم‌خوانی داشته باشد. **تست:**

```python
```

```python
def test_fact_count_matches_sequences(final_facts_file, labeled_sequences_df):
    fact_count = count_predicate_occurrences(final_facts_file, "involved_in")
    assert fact_count == len(labeled_sequences_df)
```

فاز ۴ — فرمول‌بندی دانش تاکتیکی مشترک (bk.pl)

این فاز، برخلاف فازهای قبل، **مستلزم مشارکت مستقیم و مستمر شما** است. روال هر تسک: شما مفهوم را به زبان ساده توصیف می‌کنید → با هم به Predicate منطقی تبدیل می‌کنیم → تست کوچک روی چند نمونه واقعی می‌زنیم → تأیید یا اصلاح.

تسک ۴.۱ — فرمول‌بندی مفهوم "Line-breaking Pass"

کار: تعریف مشترک اینکه یک پاس چه شرایطی داشته باشد تا "خط‌شکن" محسوب شود؛ تبدیل به قانون Prolog. **معیار پذیرش:** روی حداقل ۱۰ پاس واقعی از داده، تشخیص سیستم با تشخیص شما (به‌عنوان آنالیزور) حداقل در ۸ مورد یکسان باشد. **تست:**

python

```python
def test_line_breaking_pass_matches_expert_labels(expert_labeled_passes):
    predicted = [is_line_breaking(p) for p in expert_labeled_passes["pass_data"]]
    actual = expert_labeled_passes["expert_label"]
    accuracy = sum(p == a for p, a in zip(predicted, actual)) / len(actual)
    assert accuracy >= 0.8
```

تسک ۴.۲ — فرمول‌بندی مفهوم "تردید/مکث تحت فشار"

کار: تعریف آستانه زمانی مکث (بر اساس دانش تجربی شما) که نشان‌دهنده تردید هافبک/مدافع زیر فشار است. **معیار پذیرش:** آستانه اولیه تعیین و روی حداقل ۵ نمونه واقعی (که شما از قبل به‌عنوان "تردید محسوس" علامت زده‌اید) صحت‌سنجی شود. **تست:**

python

```python
def test_hesitation_threshold_reasonable(hesitation_labeled_samples, threshold_seconds):
    predicted = hesitation_labeled_samples["pause_duration"] > threshold_seconds
    assert (predicted == hesitation_labeled_samples["expert_label"]).mean() >= 0.7
```

تسک ۴.۳ — فرمول‌بندی مفاهیم حمایتی/رابطه‌ای (`supports`، `covers`)

کار: تعریف قوانین geometric/context-based برای اینکه چه زمانی یک بازیکن دوم "حمایت‌گر" بازیکن اول محسوب می‌شود (طبق نمونه‌های بخش ۲.۵ سند فنی). **معیار پذیرش:** روی حداقل ۱۰ نمونه دونفره واقعی، تشخیص سیستم با قضاوت شما همخوانی معقول (+٪۷۰) داشته باشد. **تست:**

```python
def test_support_relation_matches_expert(expert_labeled_pairs):
    predicted = [supports(p1, p2, seq) for p1, p2, seq in expert_labeled_pairs["pairs"]]
    accuracy = sum(p == e for p, e in zip(predicted, expert_labeled_pairs["expert_label"])) / len(expert_
    assert accuracy >= 0.7
```

تسک ۴.۴ — تجمیع نهایی bk.pl

کار: ادغام تمام Predicateهای هندسی (فاز ۳) و تاکتیکی (تسک‌های ۴.۱ تا ۴.۳) در یک فایل `bk.pl` واحد و مستند. **معیار پذیرش:** فایل کامل بدون خطا در SWI-Prolog لود شود و تمام Predicateهای موردنیاز `bias.pl` (فاز بعد) را پوشش دهد. **تست:**

```python
def test_bkpl_loads_and_covers_required_predicates(bkpl_path, required_predicates):
    result = load_prolog_file(bkpl_path)
    assert result.success
    assert required_predicates.issubset(result.defined_predicates)
```

فاز ۵ — تعریف بایاس و اجرای Popper

تسک ۵.۱ — نوشتن `bias.pl` نسخه اول (محدود)

کار: تعریف `head_pred`، `body_pred`، `max_vars`، `max_body`، و سقف تعداد بازیکنان (حداکثر ۲ برای شروع، طبق راهکار ریسک ۱ سند فنی). **معیار پذیرش:** Popper با این bias، بدون Timeout (زیر ۵ دقیقه) روی یک زیرمجموعه کوچک (یک بازی) اجرا شود. **تست:**

```python
def test_popper_runs_within_timeout_small_sample():
    result = run_popper(bk="bk.pl", bias="bias.pl", exs="exs_single_match.pl", timeout=300)
    assert result.completed
```

تسک ۵.۲ — اجرای Popper روی کل دیتاست (سطح تک‌بازیکنی)

کار: اجرای Popper با `head_pred` تک‌متغیره (`lose_possession/1`) روی کل مجموعه Fact. **معیار پذیرش:** حداقل یک قانون خروجی تولید شود؛ زمان اجرا مستند و قابل قبول باشد (زیر یک ساعت برای این حجم داده). **تست:**

```python
def test_popper_single_player_rules_generated():
    result = run_popper(bk="bk.pl", bias="bias_single.pl", exs="exs_all.pl", timeout=3600)
    assert len(result.rules) >= 1
```

تسک ۵.۳ — افزایش سقف به دوبازیکنی و اجرای مجدد

کار: تغییر `head_pred` به دومتغیره (`lose_possession/2`)، افزایش سقف بازیکنان طبق بخش ۲.۵ سند فنی، اجرای مجدد. **معیار پذیرش:** اجرا در زمان معقول (مطابق تنظیم Timeout مصوب در ریسک ۱) کامل شود؛ حداقل یک قانون دونفره تولید شود. **تست:**

```python
```

```
def test_popper_two_player_rules_generated():
    result = run_popper(bk="bk.pl", bias="bias_pairwise.pl", exs="exs_all.pl", timeout=3600)
    two_player_rules = [r for r in result.rules if r.arity == 2]
    assert len(two_player_rules) >= 1
```

فاز ۶ — فیلتر، ارزیابی و تفسیر قوانین

تسک ۶.۱ — محاسبه Support و Confidence برای هر قانون خروجی

کار: پیاده‌سازی محاسبه دو معیار طبق بخش ۶ سند فنی، روی تمام قوانین خروجی فاز ۵. **معیار پذیرش:** هر قانون یک مقدار Support و Confidence عددی داشته باشد؛ قوانین با Support کمتر از آستانه (۳ تا ۵) به‌صورت خودکار حذف شوند. **تست:**

```python
def test_support_confidence_computed_and_filtered(raw_rules_with_metrics):
    assert all("support" in r and "confidence" in r for r in raw_rules_with_metrics)
    assert all(r["support"] >= 3 for r in raw_rules_with_metrics if r["kept"])
```

تسک ۶.۲ — محاسبه Lift نسبت به Baseline تیم

کار: پیاده‌سازی معیار Lift طبق بخش ۲.۷ سند فنی، برای تفکیک الگوهای واقعاً پرخطر از الگوهای سالم پرتکرار. **معیار پذیرش:** الگوهای شناخته‌شده سالم (مثلاً کمک هافبک دفاعی، اگر در داده ظاهر شود) Lift نزدیک به ۱ داشته باشند، نه بالا. **تست:**

```python
def test_healthy_pattern_has_low_lift(known_healthy_pattern_stats):
    assert known_healthy_pattern_stats["lift"] < 1.3
```

تسک ۶.۳ — جلسه Coach Validation (تأیید انسانی)

کار: مرور دستی تمام قوانین باقی‌مانده توسط شما؛ ثبت تأیید/رد هر کدام با یادداشت دلیل. **معیار پذیرش:** هر قانون خروجی این فاز دارای وضعیت مشخص (`approved` / `rejected`) و یادداشت باشد. **تست:**

```python
def test_all_rules_have_validation_status(final_rules_df):
    assert final_rules_df["validation_status"].isin(["approved", "rejected"]).all()
```

فاز ۷ — نمایش بصری و گزارش نهایی

تسک ۷.۱ — رسم نمونه‌ای از هر قانون تأییدشده روی نقشه زمین

کار: استفاده از mplsoccer برای رسم حداقل یک نمونه واقعی از هر قانون approved، با مسیر توپ و موقعیت بازیکنان کلیدی. **معیار پذیرش:** هر قانون approved حداقل یک تصویر رسم‌شده داشته باشد. **تست:**

```python
def test_visualization_exists_for_each_approved_rule(approved_rules, output_dir):
    for rule_id in approved_rules["rule_id"]:
        assert (output_dir / f"rule_{rule_id}.png").exists()
```

تسک ۷.۲ — ترجمه قانون منطقی به توضیح قابل‌فهم

کار: پیاده‌سازی `logic_translator.py` که هر قانون Prolog را به یک جمله فارسی/انگلیسی قابل‌فهم برای مربی تبدیل می‌کند. **معیار پذیرش:** هر قانون approved یک متن توضیحی داشته باشد که حداقل شامل بازیکن(ها)، شرایط، و پیامد باشد. **تست:**

```python
```

```python
def test_translation_contains_key_elements(translated_rules):
    for t in translated_rules:
        assert t["player_mentioned"] and t["condition_mentioned"] and t["outcome_mentioned"]
```

تسک ۷.۳ — تجمیع گزارش نهایی

کار: ساخت گزارش نهایی (سند یا صفحه) شامل تمام قوانین approved، با تصویر، توضیح، و آمار (Support/Confidence/Lift). **معیار پذیرش:** گزارش نهایی شامل حداقل ۳ الگو باشد (طبق معیار موفقیت MVP در بخش ۱.۵ سند فنی). **تست:**

```python
def test_final_report_meets_mvp_criteria(final_report):
    assert len(final_report["patterns"]) >= 3
```

فاز ۸ — مستندسازی و بسته‌بندی MVP

تسک ۸.۱ — مستندسازی کد و پایپ‌لاین

کار: README کامل برای اجرای مجدد کل پایپ‌لاین از صفر، شامل نحوه اجرای هر فاز. **معیار پذیرش:** یک نفر ثالث (یا خود شما بعد از مدتی) بتواند صرفاً با خواندن README، پایپ‌لاین را از ابتدا اجرا کند.

تسک ۸.۲ — جمع‌بندی یافته‌ها و ثبت محدودیت‌های شناخته‌شده

کار: سند کوتاه جمع‌بندی: چه الگوهایی کشف شد، کدام رد شدند و چرا، چه محدودیت‌هایی (طبق بخش ۲.۴ سند فنی درباره داده 360) روی نتایج اثر گذاشت. **معیار پذیرش:** سند شامل حداقل یک بخش صریح "محدودیت‌های این نسخه" باشد.

تسک ۸.۳ — برنامه‌ریزی اولیه فاز توسعه

کار: بر اساس آنچه در MVP یاد گرفته شد (به‌خصوص سقف واقعی تعداد بازیکنان و Timeout Popper)، پیش‌نویس اهداف فاز توسعه (Set Pieces، احتمالاً Tracking Data واقعی در صورت تخصیص بودجه). **معیار پذیرش:** سند پیش‌نویس فاز توسعه آماده و برای بررسی شما ارائه شود.

نکته پایانی درباره نحوه استفاده از این فازبندی

- هر تسک را به‌ترتیب انجام دهید؛ تست pytest متناظر را بنویسید و اجرا کنید؛ فقط بعد از سبز شدن تست، تسک را تیک بزنید و به بعدی بروید.
- فاز ۴ کندتر از بقیه پیش می‌رود چون نیازمند مشارکت مستقیم شماست — این طبیعی و پیش‌بینی‌شده است (طبق ریسک ۷ سند فنی).
- اگر در هر فاز با یک مانع غیرمنتظره مواجه شدید (مثلاً Timeout مکرر Popper، یا پوشش پایین‌تر از حد انتظار داده 360)، به‌جای پیش‌روی اجباری، به‌روزرسانی سند فنی (بخش ریسک‌ها) و بازنگری در بایاس یا دامنه توصیه می‌شود.